

UVA 796

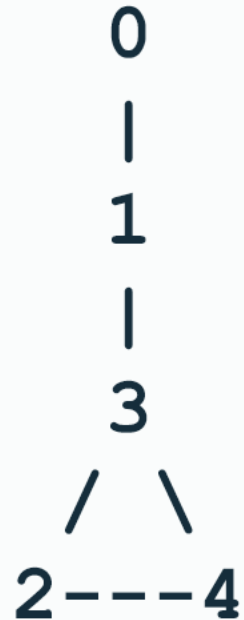
Critical Links

Implementação final e conclusão

Pedro Pinheiro Barros Leal
Emmanuel Rosendo Parente Dias

Resolução de Problemas com Grafos

Modelagem e pontes



Servidores: vértices

Conexões: arestas bidirecionais

Representação: lista de adjacência

Uma ponte aumenta o número de componentes quando removida.

Pontes: (0,1) e (1,3)

O triângulo mantém caminhos alternativos.

Referências da disciplina

Referência	Adaptação em Main.java
Graph	Graph: listas com vizinho e identidade da aresta
DepthFirstPaths	BridgeFinder: marcação, pais e exploração DFS
CC	Laço que inicia DFS em cada componente

Java, com recursos da biblioteca padrão.

Nenhuma biblioteca externa ou função pronta de pontes.

DFS com pilha explícita

O planejamento previa recursão.

A versão final armazena as chamadas em um vetor.

Estrutura	Papel
marked, parent	Visitas e pais da árvore DFS
disc, low	Descoberta e menor descoberta alcançável
stack, nextNeighbor	Caminho ativo e próximo vizinho a explorar
parentEdge	Ignorar somente a aresta de chegada

Ao esgotar os vizinhos, retirar v da pilha simula o retorno.

Critério estrutural

$$\text{low}[\text{filho}] > \text{disc}[\text{pai}]$$

A subárvore do filho não alcança o pai nem seus ancestrais por uma aresta de retorno.

No retorno: $\text{low}[\text{pai}] = \min(\text{low}[\text{pai}], \text{low}[\text{filho}])$

Vizinho marcado: $\text{low}[v] = \min(\text{low}[v], \text{disc}[w])$

A igualdade indica um caminho alternativo. A aresta não é ponte.

Rastreamento do grafo do grupo

Ordem de descoberta: 0, 1, 3, 2, 4

v	pai	disc	low final
0	-1	0	0
1	0	1	1
3	1	2	2
2	3	3	2
4	2	4	2

Aresta 4–3: $\text{low}[4] = 2$

Retorno para 2: $\text{low}[2] = 2$

(2,4): $2 > 3$ é falso

(3,2): $2 > 2$ é falso

(1,3): $2 > 1$ é verdadeiro

(0,1): $1 > 0$ é verdadeiro

Entrada, saída e adaptações

Leitura até EOF, incluindo redes com $n = 0$.

As linhas podem aparecer fora de ordem.

Adicionar uma conexão apenas quando $u < v$.

Cada caso reinicia os vetores e o contador.

Ordenar os pares pelo primeiro e depois pelo segundo extremo.

Imprimir uma linha em branco após cada caso.

A identidade da aresta evita confundir uma conexão paralela com a própria aresta-pai.

Validação local

Verificação	Resultado
6 redes do grupo e 2 da amostra oficial	Saída exata, incluindo linhas em branco
1.100 grafos pequenos e 300 aleatórios	Concordância com remoção de arestas
Entrada vazia e arestas paralelas	Resultados esperados
Caminho com 100.000 vértices	99.999 pontes, sem recursão

Testes locais passaram.

Submissão e evidência de Accepted continuam pendentes.

Complexidade e execução

Etapa	Custo
DFS e construção do grafo	$O(V + E)$
Ordenação das B pontes	$O(B \log B)$
Memória total	$O(V + E)$

A partir da pasta T2

```
javac -encoding UTF-8 -d build src/Main.java  
java -cp build Main < dados/entrada.txt
```

Conclusão e entrega

A DFS identifica as pontes sem repetir uma busca para cada aresta removida.

Solução: `src/Main.java`

Adaptações: `acompanhamento/marco-4.md`

Testes reproduzíveis: `testes/validar.py`

**Próxima etapa: revisão pelo grupo e submissão ao UVA.
Adicionar a evidência somente após um Accepted real.**